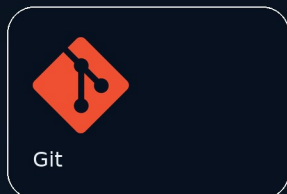


AMBIENTE E VERSIONAMENTO

GIT: FUNDAMENTOS E SEGURANÇA

Registre mudanças com intenção, leia o estado do projeto e desfça erros sem destruir trabalho.



Alexandre Carlos de Jesus

github.com/alecarlosjesus

linkedin.com/in/alexandre-carlos-de-jesus

Versões e procedimentos validados em agosto de 2026

© 2026 Alexandre Carlos de Jesus - Todos os direitos reservados.

ANTES DE COMEÇAR

Como usar esta apostila

ROTA DE ESTUDO Leia o conceito, execute cada comando, compare a saída com o quadro Confira o resultado e somente depois avance. Digitar o comando sem observar o que aconteceu não conta como prática concluída.

ENTENDA OS TERMOS ANTES DE USÁ-LOS

Palavras que você encontrará

Termo	Explicação em linguagem simples
Git	Sistema distribuído de controle de versão: registra e compara mudanças em arquivos.
Repositório	Pasta cujo histórico é controlado pelo Git.
Snapshot	Fotografia lógica do estado selecionado do projeto em um momento.
Commit	Registro identificado no histórico, com autor, data, mensagem e snapshot.
Staging area	Área de preparação que define o conteúdo do próximo commit.
Branch	Linha móvel de trabalho que aponta para uma sequência de commits.
Feature	Funcionalidade; também pode nomear uma branch criada para desenvolver uma mudança.
Merge	Operação que combina o histórico de uma branch com outra.
HEAD	Referência para o ponto do histórico em que você está trabalhando.
Hash	Identificador calculado para um objeto ou commit Git.
Working tree	Diretório de trabalho: arquivos visíveis que você edita.

O QUE SERÁ FEITO

Organização da aula

Item	Definição
Duração estimada	120 a 150 minutos
Pré-requisitos	A01 concluída ou terminal funcional; editor de código instalado.
Resultado esperado desta aula	Repositório local TechStore com histórico legível, .gitignore e branch de trabalho.
Versões e base usadas	Git 2.5x; comandos modernos switch e restore.
Arquivos trabalhados	README.md, .gitignore, docs/escopo.md e docs/checklist.md

AO FINAL DESTA AULA

O que você aprenderá

Você conseguirá definir repositório, commit, staging area, branch, HEAD e snapshot.

Você conseguirá explicar o caminho de uma mudança entre diretório de trabalho, staging area e histórico.

Você conseguirá configurar o Git, iniciar um repositório, selecionar arquivos e criar commits legíveis.

Você conseguirá interpretar git status e git diff para escolher a ação correta.

Você conseguirá escolher uma forma segura de desfazer mudanças conforme o estado do trabalho.

Você conseguirá construir o primeiro histórico local do TechStore 2026.

VEJA O ASSUNTO COMO UM TODO

Como as partes se conectam



Fluxo do Git: diretório de trabalho segue para staging area com `git add` e para repositório local com `git commit`.

HISTÓRICO COM INTENÇÃO

1. O problema que o Git resolve

Salvar projeto-final.zip, projeto-final-agora-vai.zip e projeto-final-certo-2.zip não cria um histórico confiável. Esses nomes não explicam exatamente o que mudou, quem mudou, por que mudou nem como voltar a um ponto anterior.

Git é um sistema distribuído de controle de versão. Distribuído significa que cada clone, ou cópia local completa do repositório, pode carregar todo o histórico. Nesta apostila trabalharemos somente no computador local. O GitHub aparecerá na A03 como plataforma remota de colaboração.

VOCÊ SABIA? Um commit não é apenas uma cópia de um arquivo. Ele registra um snapshot do conjunto selecionado, aponta para o commit anterior e guarda metadados, isto é, informações sobre o registro, como autor e mensagem.

Os três espaços do trabalho diário

O fluxo básico pode ser lido da esquerda para a direita: você altera arquivos no diretório de trabalho, escolhe mudanças para a staging area com `git add` e grava o conjunto preparado com `git commit`.

Espaço	Pergunta que responde	Comando de leitura
Diretório de trabalho	O que está diferente nos arquivos que edito?	<code>git diff</code>
Staging area	O que entrará no próximo commit?	<code>git diff --staged</code>
Repositório local	O que já foi gravado no histórico?	<code>git log</code>

ATENÇÃO `git add` não envia arquivos para o GitHub e não cria um commit. Ele apenas seleciona a versão atual da mudança para a staging area.

FAÇA UMA VEZ POR COMPUTADOR

2. Instalação e configuração inicial

Instalar ou validar

No Windows, use o instalador oficial Git for Windows indicado em git-scm.com/download/win. Mantenha as opções padrão do laboratório. No macOS ou Linux, siga o método oficial indicado para o sistema. Depois, abra um terminal novo.

TERMINAL

```
git --version
```

CONFIRA O RESULTADO O terminal mostra git version 2.x. O número de correção pode variar; o importante é usar uma versão moderna que inclua switch e restore.

Configurar identidade e branch inicial

A identidade fica gravada nos commits. Use seu próprio nome e um e-mail associado à conta que será usada no GitHub. Não copie o e-mail do professor.

TERMINAL - SUBSTITUA OS EXEMPLOS

```
git config --global user.name "SEU NOME COMPLETO"
git config --global user.email "seu-email@exemplo.com"
git config --global init.defaultBranch main
```

A opção --global aplica a configuração ao usuário desse computador. Para conferir os valores e descobrir de qual arquivo vieram:

TERMINAL - AUDITORIA DA CONFIGURAÇÃO

```
git config --list --show-origin
git config --global user.name
git config --global user.email
git config --global init.defaultBranch
```

ERRO COMUM Configurar user.name como o usuário do GitHub não é obrigatório e pode empobrecer a autoria. user.name é o nome exibido no commit; o vínculo com o GitHub é feito principalmente pelo e-mail reconhecido.

TECHSTORE 2026

3. Laboratório guiado: primeiro histórico

Se você concluiu A01, entre na pasta techstore-2026. Se estiver fazendo esta apostila de forma isolada, crie uma pasta vazia com esse nome.

TERMINAL

```
cd caminho/para/techstore-2026
git init
git status
```

git init cria a estrutura interna .git. Não edite manualmente essa pasta. git status informa a branch atual e lista arquivos rastreados, modificados, preparados ou ainda não rastreados.

CONFIRA O RESULTADO O terminal informa que você está na branch main e que ainda não existem commits.

Etapa 1 - criar o .gitignore antes do primeiro add

.gitignore registra padrões que o Git deve ignorar. Isso evita enviar dependências baixadas, arquivos de build, ou versão construída, registros de erro, arquivos do sistema e segredos como senhas e chaves.

.GITIGNORE

```
node_modules/
package-lock.json
dist/
.env
.env.*
!.env.example
*.log
.vscode/
.DS_Store
Thumbs.db
```

package-lock.json aparece aqui porque esta é a convenção adotada pela turma na A01. O arquivo .env costuma guardar configurações e chaves locais; por isso, o exemplo real fica fora do Git e apenas .env.example pode servir como modelo sem dados secretos.

ATENÇÃO Se um segredo já entrou em um commit, acrescentá-lo depois ao .gitignore não apaga o histórico. Revogue a credencial e trate o vazamento. .gitignore é prevenção, não borracha retroativa.

Etapa 2 - criar documentação mínima**README.MD**

```
# TechStore 2026

Projeto didatico da Colecao Vite + React + TypeScript 2026.

## Objetivo
Construir um catalogo de tecnologia por etapas.

## Autor
Seu nome
```

DOCS/ESCOPO.MD

```
# Escopo inicial

- Exibir produtos
- Consultar detalhes
- Cadastrar produtos
- Editar produtos
- Excluir produtos
```

Etapa 3 - ler antes de selecionar**TERMINAL**

```
git status --short
git diff
```

No formato curto, ?? indica arquivo não rastreado. git diff não mostra o conteúdo de arquivo ainda não rastreado, porque não existe uma versão anterior para comparar.

VAMOS PRATICAR Acrescente uma linha ao README.md e execute git status --short. Antes do primeiro commit, o arquivo continua marcado como não rastreado; não existe ainda uma versão gravada para ser chamada de modificada.

Etapa 4 - selecionar com intenção**TERMINAL**

```
git add .gitignore README.md docs/escopo.md
git status
git diff --staged
```

git diff --staged mostra exatamente o que será gravado. A staging area permite separar um trabalho grande em commits coerentes.

CONFIRA O RESULTADO Os três arquivos aparecem em Changes to be committed, ou mudanças que serão incluídas no commit.

Etapa 5 - criar o primeiro commit

TERMINAL

```
git commit -m "docs: inicia escopo do TechStore"
git status
git log --oneline --decorate
```

A mensagem começa com docs porque o commit altera documentação. Esse rótulo segue uma convenção, não uma exigência técnica do Git. A parte mais importante é descrever uma mudança concreta.

CONFIRA O RESULTADO git status mostra working tree clean, ou diretório de trabalho limpo, e git log exibe o primeiro commit.

NÃO RASTREADO, MODIFICADO, PREPARADO E GRAVADO

4. O ciclo completo de um arquivo

Crie docs/checklist.md com três critérios de conclusão. Em seguida, passe deliberadamente por cada estado e observe o terminal.

TERMINAL - EXECUTE UMA LINHA POR VEZ

```
git status --short
git add docs/checklist.md
git status --short
git diff --staged
git commit -m "docs: adiciona checklist de conclusao"
git status --short
```

Marca	Significado no status curto
??	Arquivo não rastreado.
M	Arquivo rastreado modificado apenas no diretório de trabalho.
M	Mudança preparada na staging area.
MM	Uma versão foi preparada e o arquivo recebeu outra alteração depois.
A	Arquivo novo adicionado à staging area.

VOCÊ SABIA? Em MM existem duas versões relevantes do mesmo arquivo: a versão preparada para o próximo commit e uma alteração mais nova que ainda ficou no diretório de trabalho.

GIT COMO FERRAMENTA DE INVESTIGAÇÃO

5. Ler diferenças e histórico

COMANDOS DE LEITURA

```
git diff
git diff --staged
git log --oneline --graph --decorate --all
git show HEAD
git show --stat HEAD
```

Comando	Leitura
git diff	Mudanças ainda não preparadas.
git diff --staged	Mudanças selecionadas para o próximo commit.
git log --oneline	Resumo do histórico, um commit por linha.
git show HEAD	Conteúdo e metadados do commit atual.
git show --stat HEAD	Resumo por arquivo, sem exibir todo o conteúdo.

UMA LINHA DE TRABALHO ISOLADA

6. Branches sem mistério

Branch é uma referência móvel para um commit. Criar uma branch não duplica fisicamente o projeto inteiro. Ela oferece um nome para uma linha de trabalho e avança conforme novos commits são criados nela.

No nome feature/catalogo-inicial, feature significa funcionalidade. O texto depois da barra identifica o trabalho realizado nessa branch.

TERMINAL

```
git switch -c feature/catalogo-inicial
git branch --show-current
git branch
```

Crie docs/catalogo.md com as seções Produto, Categoria e Preço. Depois grave a mudança:

TERMINAL

```
git add docs/catalogo.md
git commit -m "docs: define dados iniciais do catalogo"
git log --oneline --graph --decorate --all
```

CONFIRA O RESULTADO O histórico mostra main no commit anterior e feature/catalogo-inicial no commit mais recente.

Integre a mudança localmente:

TERMINAL

```
git switch main
git merge --no-ff feature/catalogo-inicial -m "merge: integra catalogo inicial"
git branch -d feature/catalogo-inicial
git log --oneline --graph --decorate --all
```

--no-ff solicita um commit de merge mesmo quando seria possível apenas avançar o ponteiro. Isso deixa explícito que um conjunto de commits nasceu em uma branch. A equipe pode escolher outra estratégia; o importante é ter uma regra consciente.

A PERGUNTA VEM ANTES DO COMANDO

7. Desfazer com segurança

Antes de desfazer, responda: a mudança foi preparada? Já virou commit? O commit foi compartilhado? Quero apagar conteúdo ou criar um registro que reverta o erro?

Situação	Comando	Efeito e risco
Arquivo alterado, não preparado	git restore arquivo	Descarta a alteração local do arquivo. Irreversível pelo Git se não houver outra cópia.
Arquivo preparado por engano	git restore --staged arquivo	Retira da staging area; mantém o conteúdo editado.
Último commit local com mensagem errada	git commit --amend -m "nova mensagem"	Reescreve o último commit. Use antes de publicar.
Commit já compartilhado	git revert HASH	Cria novo commit com a reversão; preserva o histórico público.
Mover ponteiro e descartar tudo	git reset --hard	Alto risco: pode apagar mudanças. Não é procedimento padrão da aula.

ATENÇÃO Nunca escolha git reset --hard porque ele parece mais rápido. Verifique git status, copie trabalho importante e confirme o alvo. Em histórico compartilhado, prefira git revert.

Simulação segura de restore

Acrescente uma linha descartável a docs/checklist.md. Observe git diff e descarte somente essa alteração:

TERMINAL

```
git diff docs/checklist.md
git restore docs/checklist.md
git status --short
```

CONFIRA O RESULTADO A linha descartável desapareceu e o arquivo voltou ao último snapshot gravado.

UM COMMIT, UMA INTENÇÃO

8. Boas mensagens e commits atômicos

Atômico significa pequeno e coerente: o commit entrega uma mudança completa que pode ser compreendida e, se necessário, revertida de forma isolada.

Os rótulos abaixo são uma convenção de escrita: docs indica documentação; fix indica correção; feat vem de feature e indica funcionalidade; chore indica manutenção. O Git não exige esses rótulos, mas a equipe pode adotá-los.

Mensagem fraca	Mensagem melhor	Por quê
update	docs: detalha campos de produto	Explica escopo e ação.
arruma tudo	fix: corrige preço negativo no cadastro	Nomeia o defeito corrigido.
mais coisas	chore: atualiza dependências do projeto	Classifica manutenção sem inventar funcionalidade.
novo	feat: adiciona filtro por categoria	Explicita a capacidade entregue.

VAMOS PRATICAR Crie uma alteração de documentação e uma correção independente. Use git add arquivo por arquivo para produzir dois commits, não um commit genérico com tudo.

VERIFIQUE O QUE APRENDEU

Exercícios

Responda antes de consultar o gabarito. Nas questões práticas, registre o comando usado e a saída observada.

1. Defina diretório de trabalho, staging area e repositório local.

Resposta: _____

2. Explique por que git add não significa salvar no GitHub.

Resposta: _____

3. Liste os comandos para criar um repositório com branch inicial main, preparar README.md e criar o primeiro commit.

Resposta: _____

4. O status curto mostra MM README.md. O que isso revela?

Resposta: _____

5. Você adicionou .env à staging area, mas ainda não criou commit. Como retirar o arquivo da staging sem apagar seu conteúdo?

Resposta: _____

6. Um commit com erro já foi enviado para outras pessoas. Entre amend, revert e reset --hard, qual opção preserva melhor o histórico compartilhado?

Resposta: _____

7. Crie uma branch docs/arquitetura, faça um commit coerente, integre-a à main e remova a branch.

Resposta: _____

8. Por que colocar node_modules no repositório prejudica o projeto?

Resposta: _____

9. Reescreva a mensagem 'atualizações finais' para um cenário em que foi corrigida a validação de e-mail.

Resposta: _____

10. Gere um log gráfico do TechStore e identifique visualmente main, o merge e os dois primeiros commits.

Resposta: _____

APRENDA TAMBÉM COM A CORREÇÃO

Gabarito comentado

1. Diretório de trabalho é onde os arquivos são editados; staging area seleciona o conteúdo do próximo commit; repositório local armazena os snapshots já gravados.
2. git add move uma versão da mudança para a staging area local. Não há comunicação remota. O envio para GitHub será feito posteriormente por git push.
3. git init; conferir git branch --show-current; se necessário git branch -M main; git add README.md; git diff --staged; git commit -m "docs: inicia projeto".
4. Há uma versão modificada do README já preparada e outra modificação mais nova ainda não preparada. git diff --staged mostra a primeira; git diff mostra a segunda.
5. git restore --staged .env. Depois, acrescente .env ao .gitignore e confirme com git status.
6. git revert HASH, porque cria outro commit que desfaz o efeito sem reescrever o histórico que outras pessoas já receberam.
7. git switch -c docs/arquitetura; editar; git add; git commit; git switch main; git merge --no-ff docs/arquitetura; git branch -d docs/arquitetura.
8. node_modules é grande e pode ser recriado com npm install a partir das dependências registradas no package.json. Versioná-lo aumenta o tamanho e os conflitos sem necessidade. Nesta turma, package-lock.json também fica fora do Git conforme a convenção definida na A01.
9. Exemplo: fix: corrige validação de e-mail no cadastro. A mensagem informa o tipo e o efeito da mudança.
10. git log --oneline --graph --decorate --all. A resposta deve apontar os nomes das referências ao lado dos hashes e o desenho do merge.

ANTES DE ENCERRAR

Lista de verificação final

- Consigo explicar working tree, staging area e repositório local;
- Consigo configurar identidade e branch main;
- Consigo usar status e diff antes de add e commit;
- Consigo escrever commits pequenos e descritivos;
- Consigo criar, integrar e remover uma branch local;
- Consigo escolher restore, amend ou revert conforme o estado da mudança;
- Consigo proteger dependências, builds e segredos com .gitignore.

BIBLIOGRAFIA E ATUALIZAÇÃO

Referências oficiais

- [Pro Git - configuração inicial](#)
- [Pro Git - gravando mudanças](#)
- [Pro Git - desfazendo mudanças](#)
- [Pro Git - branches e merges](#)

Documentação consultada e procedimentos validados em 10 de agosto de 2026.

AUTORIA E DIREITOS | Alexandre Carlos de Jesus • GitHub: [alecarlosjesus](#) • LinkedIn: [alexandre-carlos-de-jesus](#) • © 2026
Alexandre Carlos de Jesus. Todos os direitos reservados. Reprodução, distribuição, adaptação ou comercialização, total ou parcial, somente com autorização prévia por escrito, salvo permissões legais. Marcas e logotipos pertencem aos respectivos titulares.